

Cours d'introduction à la Data Science

Chargé du cours : Jules DEGILA, PhD, MCF
Assistante : Aurélie KPOZE, PhD

jules.degila@imsp-uac.org, satou.kpoze@imsp-uac.org

23 Janvier 2026



Partie 6: Architectures profondes

Différents types de couches en Deep Learning

- Un réseau de neurones profond est composé de différents types de couches, chacune ayant un rôle spécifique.
- **Couche entièrement connectée (Dense) :**
 - Chaque neurone est connecté à tous les neurones de la couche précédente.
 - Très utilisée dans les MLP et en fin de CNN.
- **Couche de convolution :**
 - Extrait des caractéristiques locales à l'aide de filtres.
 - Spécialisée pour les images (CNN).
- **Couche de pooling :**
 - Réduit la taille des données (sous-échantillonnage).
- **Couche récurrente (RNN, LSTM, GRU) :**
 - Traite des données séquentielles avec une mémoire.
- **Autres couches :** BatchNorm, Dropout, etc.

CNN : Convolutional Neural Networks

- Les CNN sont des réseaux de neurones spécialisés pour le traitement des images et données spatiales.
- Ils exploitent la structure locale des données grâce aux opérations de convolution.
- Un CNN est généralement composé de :
 - Couches de convolution
 - Fonctions d'activation (souvent ReLU)
 - Couches de pooling (sous-échantillonnage)
 - Couches entièrement connectées (classification)
- Avantages :
 - Moins de paramètres qu'un MLP classique
 - Invariance partielle aux translations
 - Excellentes performances en vision par ordinateur

Opération de convolution

Définition

L'opération de convolution consiste à **faire glisser un filtre (kernel)** sur l'entrée et à calculer, à chaque position, un **produit scalaire** entre le filtre et un patch de l'image. Le résultat est une **carte de caractéristiques (feature map)**.

Formule (2D, 1 canal):

$$Y[i, j] = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K[u, v] \cdot X[i \cdot s_h + u - p_h, j \cdot s_w + v - p_w] + b$$

- X : entrée (image ou carte)
- K : kernel (filtre)
- (s_h, s_w) : stride (pas de déplacement)
- (p_h, p_w) : padding (zéros ajoutés autour)
- b : biais
- Y : feature map de sortie

Entrée X

- Cas 1 canal (image en niveaux de gris) :

$$X \in \mathbb{R}^{H \times W}$$

- Cas multi-canaux (ex : RGB) :

$$X \in \mathbb{R}^{H \times W \times C_{in}}$$

- H : hauteur de l'image W : largeur de l'image C_{in} : nombre de canaux d'entrée
- Chaque élément $X[i, j]$ ou $X[i, j, m]$ représente une **intensité de pixel**

Filtre (Kernel) et Stride

Filtre (Kernel)

- Cas 1 canal :

$$K \in \mathbb{R}^{k_h \times k_w}$$

- Cas multi-canaux :

$$W \in \mathbb{R}^{k_h \times k_w \times C_{in} \times C_{out}}$$

- k_h, k_w : hauteur et largeur du filtre
- C_{out} : nombre de filtres = nombre de cartes de sortie
- Chaque filtre détecte un **motif particulier**

Stride (pas de déplacement)

- Noté : (s_h, s_w)
- Définit de combien de pixels le filtre se déplace horizontalement et verticalement
- Grand stride $\Rightarrow$ sortie plus petite (sous-échantillonnage)

Padding

- Noté : (p_h, p_w)
- Ajoute des **zéros autour de l'image**
- Permet de contrôler la taille de la sortie et le traitement des bords

Biais

- Noté : b ou $b[c]$
- Un biais par **canal de sortie**
- Ajouté à chaque pixel de la feature map :

$$Y = X * W + b$$

Le Kernel (Filtre) dans un CNN

Définition

Un **kernel** (ou **filtre**) est une petite matrice (ex. 3×3) qui :

- se **déplace** sur l'image (glissement),
- calcule un **produit scalaire** avec chaque petit patch,
- produit une **feature map**,
- détecte un **motif** (bord, coin, texture, ...).

Entrée (image 5×5)

$$X = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Kernel (bord vertical)

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Le Kernel (Filtre) dans un CNN

Calcul sur la frontière :

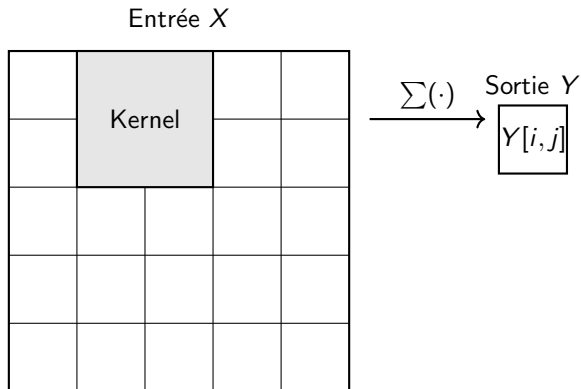
$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$$

$$\Rightarrow Y = (1 \cdot 1 + 1 \cdot 0 + 0 \cdot (-1)) \times 3 = 3$$

Interprétation

Valeur forte $\Rightarrow$ le kernel a **détection un bord vertical** à cet endroit.

Schéma (glissement du kernel)



Le kernel se déplace (stride)
et calcule un produit scalaire à chaque position.

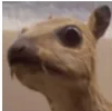
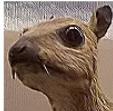
Exemple numérique (sans padding, stride 1)

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

À la position $(i, j) = (0, 0)$:

$$Y[0, 0] = 1 \cdot 1 + 0 \cdot 2 + 0 \cdot 4 + (-1) \cdot 5 = -4$$

Exemples de filtres

<i>Original</i>	<i>Gaussian Blur</i>	<i>Sharpen</i>	<i>Edge Detection</i>
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

Hyperparamètres clés

Paramètre	Effet
Kernel size (k_h, k_w)	Champ réceptif local, motifs capturés
Stride (s_h, s_w)	Sous-échantillonnage spatial (plus grand $\Rightarrow$ plus petit)
Padding (p_h, p_w)	Contrôle bordures et taille de sortie
C_{out} (nb filtres)	Profondeur de sortie / nb de features

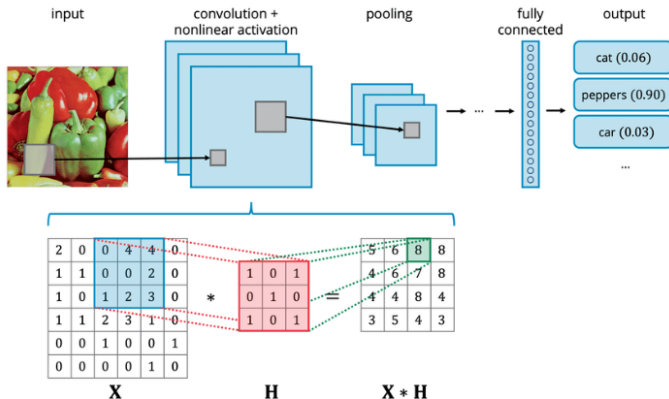
Pourquoi la convolution ?

- **Partage des poids** : beaucoup moins de paramètres qu'une couche fully-connected.
- **Localité** : capture des motifs locaux (bords, textures, formes).
- **Équivariance à la translation** : déplacer l'entrée déplace la carte de sortie.

Pipeline typique d'une couche

- ① Convolution : $X \xrightarrow{\text{conv}} Z$
- ② Biais : $Z + b$
- ③ Activation (ex. ReLU) : $A = \max(0, Z + b)$
- ④ (Optionnel) Pooling / normalisation

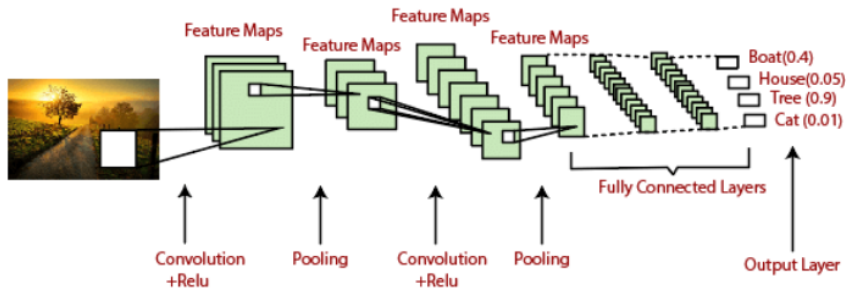
Pipeline typique d'une couche



Basic CNN architecture and kernel. A typical CNN consists of several component types (e.g., several convolutional + nonlinear activation layers, interleaved with pooling layers, followed by fully connected layers before prediction). The convolutional kernel sums the point-wise multiplications of a subset of the signal $1 \times X$ with a learned filter H . This operation is repeated as the filter "slides" across the input. (Note that what is termed convolution by the deep learning community is actually cross-correlation.)

- Applications : Classification d'images, détection d'objets, segmentation, reconnaissance faciale

CNN : Convolutional Neural Networks



RNN : Recurrent Neural Networks

- Les RNN sont des réseaux de neurones conçus pour traiter des données séquentielles.
- Ils sont utilisés pour le texte, la parole et les séries temporelles.
- Un RNN possède une mémoire interne qui permet de prendre en compte les informations passées.
- À chaque instant t , l'état caché est mis à jour :

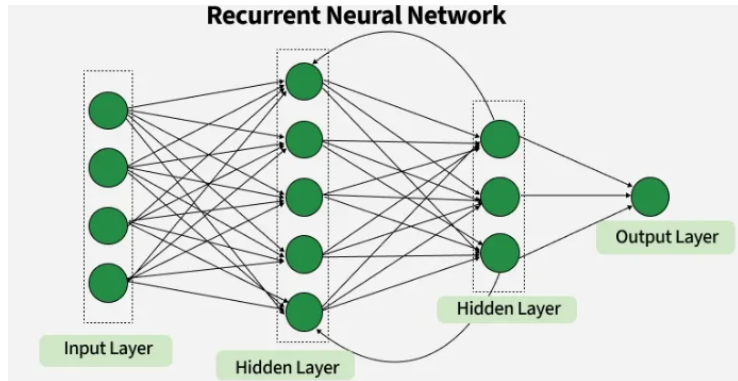
$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

- Avantages :
 - Capables de modéliser des dépendances temporelles
 - Partage des poids dans le temps
- Limite :
 - Sensibles au problème du vanishing gradient (d'où LSTM)

- Applications : Traitement du langage (NLP); reconnaissance vocale; séries temporelles

RNN : Recurrent Neural Networks

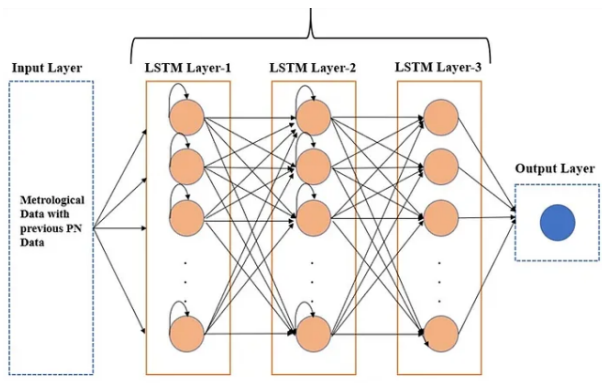
Recurrent Neural Network



LSTM : Long Short-Term Memory

- Les LSTM sont une amélioration des réseaux récurrents (RNN).
- Ils ont été conçus pour résoudre le problème du **vanishing gradient**.
- Leur objectif est de mieux mémoriser l'information sur de longues séquences.
- Contrairement aux RNN simples, un LSTM possède :
 - un **état de mémoire** (cell state)
 - des **portes de contrôle** (gates) qui décident quoi : oublier, conserver et ajouter
- Les trois portes principales sont : porte d'oubli, porte d'entrée et porte de sortie
- Avantages :
 - Capable de capturer des dépendances à long terme
 - Beaucoup plus stable que les RNN classiques

LSTM : Long Short-Term Memory



GRU : Gated Recurrent Unit

- Le GRU est une variante améliorée des réseaux récurrents (RNN).
- Il a été proposé pour résoudre le problème du **vanishing gradient**.
- Contrairement au LSTM, le GRU est plus simple :
 - Il ne possède pas d'état mémoire séparé.
 - Il utilise seulement **deux portes** :
 - Porte de mise à jour (Update gate)
 - Porte de réinitialisation (Reset gate)
- Avantages :
 - Moins de paramètres que le LSTM
 - Plus rapide à entraîner
 - Très bonnes performances en pratique
- Les GRU sont très utilisés en NLP, audio et séries temporelles.

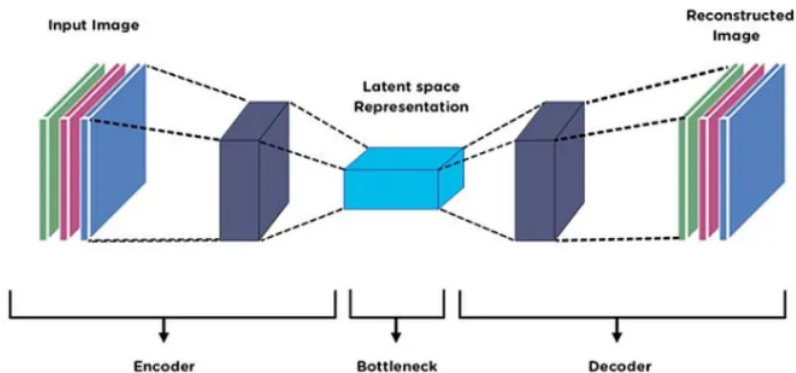
Architecture Encoder–Decoder

- L'architecture **Encoder–Decoder** est utilisée pour transformer une séquence en une autre.
- Elle est très utilisée en :
 - traduction automatique
 - résumé de texte
 - reconnaissance vocale
- **Encodeur :**
 - lit la séquence d'entrée
 - la transforme en une représentation interne (vecteurs de caractéristiques)
- **Décodeur :**
 - génère la séquence de sortie
 - en utilisant la représentation produite par l'encodeur
- Schéma conceptuel :

Entrée → Encodeur → Représentation → Décodeur → Sortie

- Cette architecture est utilisée aussi bien avec RNN / LSTM que les transformers

Architecture Encoder-Decoder



- Les RNN simples souffrent de plusieurs limitations :
- **Vanishing gradient :**
 - Le gradient disparaît lorsqu'on remonte loin dans le temps.
 - Le réseau oublie les informations anciennes.
- **Exploding gradient :**
 - Le gradient devient très grand, rendant l'apprentissage instable.
- **Calcul séquentiel :**
 - Les RNN ne sont pas parallélisables efficacement.
 - Entraînement lent sur de longues séquences.
- Ces problèmes ont motivé LSTM, GRU et plus tard les Transformers.

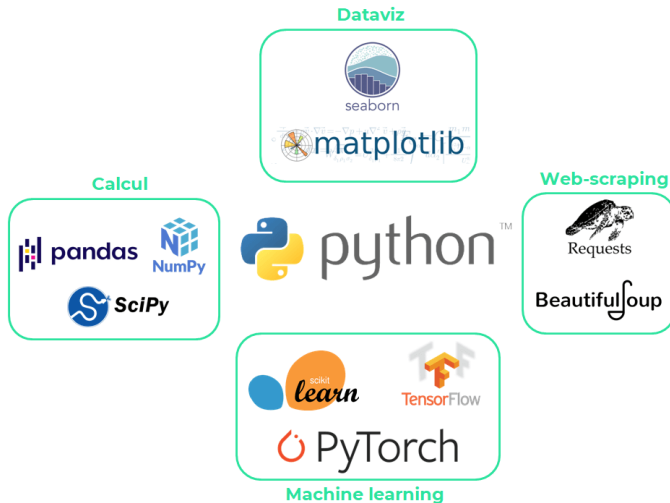
Problème : réseaux très profonds

- Plus le réseau est profond, plus :
 - le gradient disparaît / explose
 - l'optimisation devient difficile
- Phénomène observé : **dégradation des performances**

Travaux Pratiques

- **PyTorch** : dynamique, recherche/enseignement, très utilisé en production aussi
- **TensorFlow/Keras** : historique, production, écosystème large
- **JAX** : très performant, compilation (XLA), recherche
- **Ecosystèmes** :
 - vision : torchvision, timm
 - texte : torchtext (attention compat), transformers
 - audio : torchaudio

Panorama des bibliothèques Deep Learning



Qu'est-ce qui coûte cher en Deep Learning ?

Le DL repose sur des opérations très fréquentes :

- **multiplications matrice-matrice** (GEMM) : couches linéaires, attention
- **convolutions** : CNN
- **opérations tensor** massivement parallèles (softmax, normalisation, etc.)

Idée clé : beaucoup d'opérations simples sur beaucoup de données $\Rightarrow$ **parallélisme**.

CPU

- 4–32 cœurs (souvent), très puissants
- optimisé pour la diversité de tâches et le contrôle
- caches sophistiqués, bonne latence

GPU

- des milliers de cœurs plus simples
- optimisé pour les opérations **parallèles** et **débit** (throughput)
- excellent pour GEMM / convolutions

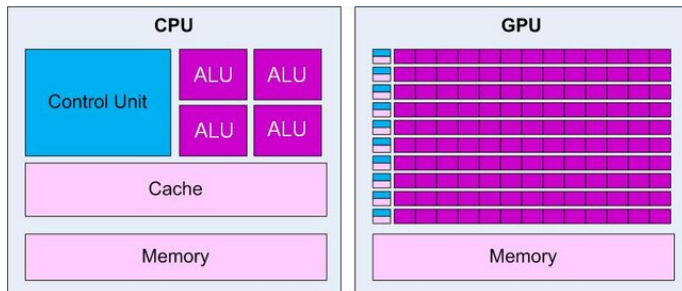
Quand le GPU n'aide pas (ou peu) ?

Le GPU n'est pas magique :

- petits modèles / petits batchs : overhead de lancement kernels + transferts
- pré-traitement lourd CPU (tokenisation, augmentation complexe)
- modèles très **branchés** (beaucoup de `if`) : faible parallélisme

Règle : si le calcul est dominé par des grosses multiplications, le GPU gagne.

CPU vs GPU



CUDA c'est quoi ?

CUDA = plateforme + API NVIDIA pour programmer le GPU.

- expose un modèle de calcul parallèle (threads, blocs, grilles)
- fournit un runtime et des bibliothèques hautement optimisées
- utilisé *indirectement* par PyTorch/TF (vous n'écrivez pas forcément de CUDA)

Alternative : AMD / ROCm (1 slide)

Pour GPU AMD, l'écosystème se base sur **ROCm**.

- support selon modèles GPU et versions
- PyTorch peut être compilé/installé avec ROCm

Dans ce TP, on se concentre sur **CUDA** (NVIDIA), le cas le plus courant.

Mémoire GPU : le vrai goulot d'étranglement

- la VRAM (mémoire GPU) est limitée (ex: 8/12/24 Go)
- un modèle = paramètres + gradients + états optimiseur + activations
- les activations (intermédiaires) peuvent dominer en entraînement

Symptôme : erreur CUDA out of memory.

Réduire la mémoire : techniques

- diminuer **batch size**
- utiliser **mixed precision** (FP16/bfloat16) : souvent 2x moins de VRAM
- **gradient checkpointing** (recalcul partiel)
- modèles plus petits / pruning / moins de couches

Transferts CPU $\leftrightarrow$ GPU : attention

- transférer les données coûte cher (PCIe)
- si vous transférez à chaque petite opération : **très lent**

Bonne pratique :

- mettre les batches sur GPU **une fois** par itération
- éviter `.cpu()` / `.numpy()` dans la boucle d'entraînement

PyTorch : détecter et utiliser le bon device

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

model = MyModel().to(device)

for x, y in loader:
    x = x.to(device)
    y = y.to(device)
    ...
```

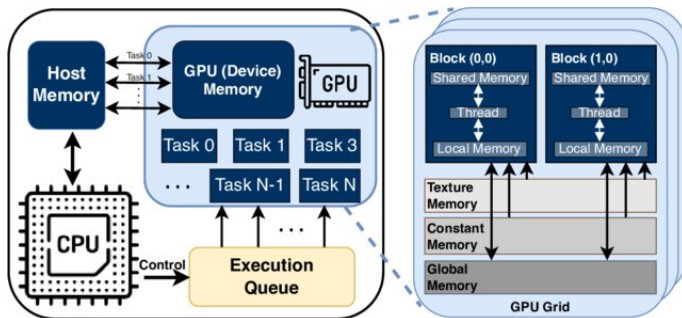
Piège classique : modèle et données sur des devices différents

- Erreur typique : Expected all tensors to be on the same device

```
# Mauvais : model sur GPU, x sur CPU
model.to("cuda")
x = torch.randn(32, 10)      # CPU
y = model(x)                 # ERREUR
```

Solution : déplacer **aussi** les données.

Modèle de calcul GPU/CPU



- ① 01_tenseurs_gradients_exercices.ipynb
- ② 02_fondations_deep_learning.ipynb
- ③ 03_TP_CNN_RNN_GRU